

VECTORS AND MATRICES

Chapter 2

Copyright © 2017 STEM Course Prep



MATRICES

10	15.7	23.1
20	18.2	25.8
30	19.2	26.9
40	19.8	28.3

- ❖ Experimental data is recorded in tabular format as shown to the left
- ❖ There are rows and columns of data
- ❖ This gives a **matrix**

- ❖ Matrices contain values of the same type
- ❖ Each value in the matrix is stored in an element
- ❖ The size of a matrix is noted by the number of rows and the number of columns
 - ❖ For m rows and n columns, we say 'm by n' or write m x n
 - ❖ Number of rows always goes first!
 - ❖ Matrix above: 4 rows, 3 columns → 4 x 3 matrix
- ❖ Matrices are also known as **arrays**

VECTORS AND SCALARS

- A set of data with only one row or one column is a **vector**
- Vectors are simply matrices that have one of the dimensions equal to 1
- A row of data is known as a **row vector**

10	23	4	16	25
----	----	---	----	----

- 1 x 5 matrix shown above. Notice the number of rows = 1
- A column of data is known as a column vector

8
5
1

- 3 x 1 matrix shown here. Notice the number of columns = 1

- A 1 x 1 matrix is just a single value

21

 - Known as a **scalar**

CREATING ROW VECTORS

- Two methods: direct or colon operator
- Direct: Values contained in square brackets, separated by commas or spaces

```
Command Window
>> row_vec=[1, 3, 6, 8]

row_vec =

     1     3     6     8

>> row_vec=[2 4 6 8]

row_vec =

     2     4     6     8
```

CREATING ROW VECTORS

5

```
>> n=5:5:20
```

```
n =
```

```
    5    10    15    20
```

```
>> nn=1:5
```

```
nn =
```

```
    1    2    3    4    5
```

```
>> nn_rev=5:-1:1
```

```
nn_rev =
```

```
    5    4    3    2    1
```

```
>> |
```

Colon operator

- Creates values in the form **first:step:last**
- Example: To get a vector that starts at 5, ends at 20, in steps of 5, use: 5:5:20
- If a step size is not specified, a step size of 1 will be used
- To decrement numbers use a step size of -1

Linspace

- **linspace** is used when evenly spaced data points are needed
- Usage: **linspace(x, y, n)**
 - Gives n data points in the range of x to y
 - If n is omitted 100 data points are created
 - **linspace(x, y)**

```
>> vel=linspace(20,30,4)

vel =

    20.0000    23.3333    26.6667    30.0000
```

```
>> |
```

LOGSPACE

- **logspace** creates a logarithmically spaced vector
- Usage: **logspace(x,y,n)**
 - Gives n data points in the range of 10^x to 10^y
 - If n is omitted 50 data points generated

```
>> logspace(1,2,3)

ans =

    10.0000    31.6228   100.0000
```

CONCATENATION

- **Concatenation** – joining vectors together or adding additional elements to an existing vector
- List vectors/numbers you want to combine inside square brackets

```
>> vec1=1:2:5;  
>> vec2=3:4;  
>> vec3=[vec1 vec2]
```

```
vec3 =
```

```
     1     3     5     3     4
```

```
>> vec4=[vec3 8 9]
```

```
vec4 =
```

```
     1     3     5     3     4     8     9
```

REFERRING TO ELEMENTS

- Values in a matrix/vector are stored as elements
- Vector elements are numbered sequentially from left to right or top to bottom
- The value of an element's row or column number is the **index**
- Use the index to refer to elements – this allows individual matrix/vector values to be used

The diagram illustrates how elements in a matrix or vector are indexed. It shows two tables. The first table is a 1x5 matrix with values 10, 12, 14, 16, and 18. The second table is a 3x2 matrix with values 7, 9, and 13 in the first column, and 10, 12, 14, 16, and 18 in the second column. In both tables, the first column is highlighted with a pink oval, and the text 'index numbers' is written next to it with arrows pointing to the highlighted areas.

1	2	3	4	5
10	12	14	16	18

index numbers

1	7
2	9
3	13

REFERRING TO ELEMENTS

```
>> vec=1:6

vec =

     1     2     3     4     5     6

>> vec(3)

ans =

     3

>> vec(5)

ans =

     5

>> vec([3 5])

ans =

     3     5

>> vec([2:5])

ans =

     2     3     4     5
```

- To use the index, type the vector name followed by the index in parenthesis
- To reference more than one element use a **index vector**
 - Index vector lists the element numbers in brackets
 - Colon operator can be used

MODIFYING ELEMENTS

- ❖ Once vector has been created, individual values can be changed using index
- ❖ Need to add data to your vector?
 - ❖ Extend the vector to include new data
 - ❖ Notice gaps in data are filled in with zeros
 - ❖ This method is inefficient!

```
Command Window
>> vec=1:3

vec =

     1     2     3

>> vec(2)=100

vec =

     1    100     3

>> vec(5:6)=[10 20]

vec =

     1    100     3     0    10    20
```

COLUMN VECTORS

- $m \times 1$ vector is a column vector
- Two methods to create: Direct and colon operator with transpose
- **Direct Method:** values are separated by semi-colons in square brackets
 - Semi-colons indicate when to move to next row

Command Window

```
>> column=[1; 3; 5]
```

```
column =
```

```
1
```

```
3
```

```
5
```

COLUMN VECTORS ¹³

- Colon operator is used to create row vectors
- For column vectors, simply create a row vector and then transpose it
 - To transpose use transpose function or the apostrophe '

```
>> row=1:2:5

row =

     1     3     5

>> col=transpose(row)

col =

     1
     3
     5
|
>> col=row'

col =

     1
     3
     5
```

COLUMN VECTORS

Index used to
refer to elements

```
num =  
  
    20  
    30  
    40  
  
>> num(1)  
  
ans =  
  
    20
```

Index used to
change elements

```
num =  
  
    20  
    30  
    40  
  
>> num(3)=1500  
  
num =  
  
    20  
    30  
    1500
```

CREATING MATRICES

- Will have m rows and n columns – $m \times n$ matrix
 - Values within rows separated by spaces or commas
 - Direct method or colon method can be used to form rows
 - Must have same number of elements in each row!!!
 - Semi-colons separate the rows

```
Command Window
>> matrix=[1 7 9;2:2:6;10 11 12]

matrix =

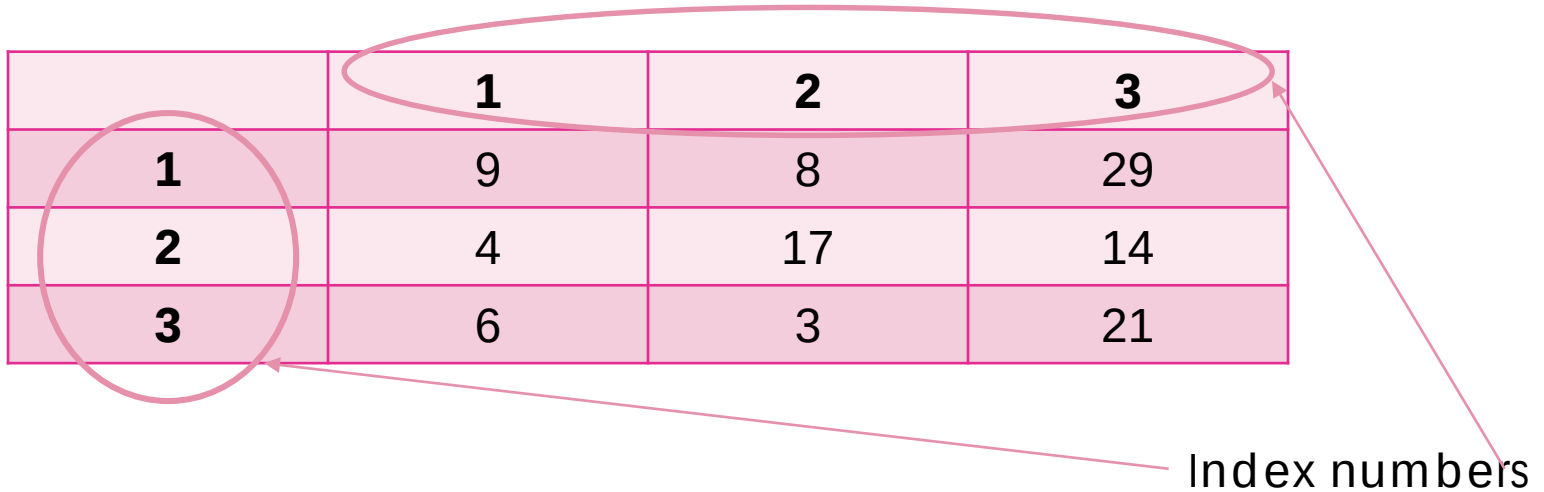
     1     7     9
     2     4     6
    10    11    12
```

CREATING MATRICES

- Many built-in functions to create matrices
 - Random number generators:
 - **rand(n)** - generates square $n \times n$ matrix of random real numbers in the open interval (0,1)
 - **rand(n,m)** - generates $n \times m$ matrix of random real numbers in the open interval (0,1)
 - **randi([range],n,m)** - generates $n \times m$ matrix of random integers in the range specified by the closed interval
 - Zeros and ones:
 - **zeros(n)** - generates square $n \times n$ matrix of zeros
 - **zeros(n,m)** - generates $n \times m$ matrix of zeros
 - **ones(n)** - generates $n \times n$ matrix of ones
 - **ones(n,m)** - generates $n \times m$ matrix of ones
 - Why might we want this? We'll cover it later on

MATRIX ELEMENTS¹⁷

- Subscripted indexing – referring to an element of the matrix
 - Row first, column second – ALWAYS!



A 3x3 matrix is shown with row and column index numbers circled in pink. A pink oval highlights the first column's index numbers (1, 2, 3), and another pink oval highlights the first row's index numbers (1, 2, 3). A pink arrow points from the text 'Index numbers' to the first row's index numbers.

	1	2	3
1	9	8	29
2	4	17	14
3	6	3	21

`mat(1,1)=9`

`mat(1,2)=8`

`mat(1,3)=29`

`mat(2,1)=4`

`mat(2,2)=17`

`mat(2,3)=14`

`mat(3,1)=6`

`mat(3,2)=3`

`mat(3,3)=21`

MATRIX ELEMENTS

- Use a colon (:) to refer to an entire row or column
 - : indicates all rows or columns
- $\text{mat}(m, :)$ – refers to the entire m^{th} row
 - : indicates all columns – all columns of the m^{th} row
- $\text{mat}(:, n)$ – refers to entire n^{th} column
 - : indicates all rows – all rows of the n^{th} column

	1	2	3
1	9	8	29
2	4	17	14
3	6	3	21

$\text{mat}(1, :) = 9 \quad 8 \quad 29$
 $\text{mat}(2, :) = 4 \quad 17 \quad 14$
 $\text{mat}(3, :) = 6 \quad 3 \quad 21$

$\text{mat}(:, 1) = 9$
 4
 6

$\text{mat}(:, 2) = 8$
 17
 3

$\text{mat}(:, 3) = 29$
 14
 21

MATRIX INDEXING ¹⁹

- Modify subsets of matrix using assignment statements

$$B = \begin{bmatrix} 12 & 0 & 0 \\ 102 & 108 & 0 \\ 0 & 30 & 120 \end{bmatrix}$$

$$B(:, 3) = [10 \ 20 \ 30]' \quad \text{gives} \quad B = \begin{bmatrix} 12 & 0 & 10 \\ 102 & 108 & 20 \\ 0 & 30 & 30 \end{bmatrix}$$

$$B(1, :) = 5:5:15 \quad \text{gives} \quad B = \begin{bmatrix} 5 & 10 & 15 \\ 102 & 108 & 20 \\ 0 & 30 & 30 \end{bmatrix}$$

- **end** will give the last row or column of a matrix

$$B(\text{end}, 2) = 30 \quad B(1, \text{end}) = 15$$

MATRIX DIMENSIONS

- Functions to determine the dimensions of a vector or matrix:
 - **length**
 - Vector: returns the # of elements
 - Matrix: returns the larger dimension (row or column)
 - Usage: **length(name)**
 - **numel**
 - Returns number of elements in vector or matrix
 - Usage: **numel(name)**
 - **size**
 - Returns the number of rows and columns for a vector or matrix
 - Returns two values – # of rows, # of columns
 - Must assign variables to both these values
 - Usage: **[r c] = size(mat)**

r = # of rows c = # of columns



MATRIX DIMENSIONS

- Examples:

$$A = \begin{bmatrix} 7 & 5 & 3 \\ 9 & 5 & 1 \end{bmatrix}$$

`length(A)=3`

`numel(A)=6`

`[r c]=size(A)` `r=2` `c=3`

$$C = [6 \quad 4 \quad 3 \quad 8 \quad 7]$$

`length(C)=5`

`numel(C)=5`

`[r c]=size(A)` `r=1` `c=5`

CHANGING DIMENSIONS

- Functions to change matrix dimensions
 - **reshape** – converts a matrix to a new matrix with different dimensions but the same number of elements

$$A = \begin{bmatrix} 1 & 5 & 3 & 4 \\ 8 & 5 & 7 & 18 \\ 6 & 12 & 23 & 14 \end{bmatrix}$$

- A has 12 elements
- reshape can be used to put these elements into matrices with the following dimensions: 1x12, 12x1, 4x3, 2x6, 6x2
- Works through the matrix columnwise
- `reshape(A, 1, 12)` gives a 1x12 matrix

CHANGING DIMENSIONS

- Other functions:
- **rot90** rotates a matrix 90 degrees counter-clockwise
- **fliplr** flips columns of a matrix from left to right (about a vertical line)
- **flipud** flips rows of a matrix up to down (about a horizontal line)
- **repmat** replicates a matrix; creates $m \times n$ copies of the matrix

ARRAYS AS FUNCTION ARGUMENTS

- Let's say we have a 1x10 array of data and we need to find the square root of each element in the array
 - Easy to do in MATLAB
 - Use the array as a function argument
 - Operation will be performed on every element

```
array=randi([1,100],1,10)
```

```
array =
```

```
82    91    13    92    64    10    28    55    96    97
```

```
sqrt(array)=  9.0554    9.5394    3.6056    9.5917    8.0000
             3.1623    5.2915    7.4162    9.7980    9.8489
```

MIN & MAX

25

max - finds the max value in a vector or in a matrix column

```
>> nums=[1 -3 5 8 -4];  
>> max(nums)  
ans =  
      8  
  
>> A=[1 5 -9;3 -1 15]  
A =  
      1      5     -9  
      3     -1     15  
  
>> max(A)  
ans =  
      3      5     15
```

min – finds the min value in a vector or the in matrix column

```
>> nums=[1 -3 5 8 -4];  
>> min(nums)  
ans =  
     -4  
  
>> A=[1 5 -9;3 -1 15]  
A =  
      1      5     -9  
      3     -1     15  
  
>> min(A)  
ans =  
      1     -1     -9
```

MIN & MAX

- To find max/min of entire matrix use function twice

```
>> B=randi([-10,10],2,4)
```

```
B =
```

```
    -7    10     6    -2
    10     0    -8     9
```

```
>> m=max(B)
```

```
m =
```

```
    10    10     6     9
```

Using **max** results in a vector

```
>> maxB=max(m)
```

```
maxB =
```

```
    10
```

Using **max** again results in a scalar that is the max value of the matrix

```
>> maxB2=max(max(B))
```

More efficient way to do this

```
maxB2 =
```

```
    10
```

SUM & CUMSUM

27

sum – returns the sum of all elements in a vector or matrix column

```
C =  
      2      4      6      8
```

```
>> sum(C)
```

```
ans =  
      20
```

```
matC =  
      1      4      7     10  
      3      5      7      6
```

```
>> sum(matC)
```

```
ans =  
      4      9     14     16
```

```
>> sum(sum(matC))
```

```
ans =  
      43
```

Copyright © 2017 STEM Course Prep

cumsum (cumulative sum) – gives the running sum of vector elements or matrix column

```
D =  
      3      6      9     12
```

```
>> cumsum(D)
```

```
ans =  
      3      9     18     30
```

```
matD =  
      6     -10      4  
     10      7      5  
      3      9      5
```

```
>> cumsum(matD)
```

```
ans =  
      6     -10      4  
     16      -3      9  
     19      6     14
```

4+5

9+5

PROD & CUMPROD

28

prod - finds product of elements in a vector or matrix column

```
>> A=2:4:14
A =
     2     6    10    14
>> prod(A)
ans =
    1680
```

$2 \times 6 \times 10 \times 14$

```
>> matA=randi([1,10],3,4)
matA =
     4     8     1     7
     7     1     1     4
     2     3     9    10
>> prod(matA)
ans =
    56    24     9    280
```

$7 \times 4 \times 10$

cumprod – finds the running product of elements in a vector or matrix column

```
>> A=2:4:14
A =
     2     6    10    14
>> cumprod(A)
ans =
     2    12   120   1680
```

12×10

```
>> matA=randi([1,10],3,4)
matA =
     4     8     1     7
     7     1     1     4
     2     3     9    10
>> cumprod(matA)
ans =
     4     8     1     7
    28     8     1    28
    56    24     9   280
```

7×4

28×10

SCALAR OPERATIONS

- Need to perform an operation on every element in a matrix?
- Easy to do using math operators

```
>> matnums=randi([-5,5],2,5)
matnums =
    -3    -3     2     5    -4
     3     0     4     1    -4
>> matnums*-1
ans =
     3     3    -2    -5     4
    -3     0    -4    -1     4
>> matnums+10
ans =
     7     7    12    15     6
    13    10    14    11     6
```

MATRIX MATH

- o Matrix Addition & Subtraction
 - o Performed element by element
 - o Must have the same dimensions in each matrix

```
>> A=randi([1,5],2,2)
```

```
A =
```

```
    2    2
```

```
    5    5
```

```
>> B=randi([1,5],2,2)
```

```
B =
```

```
    2    2
```

```
    5    1
```

```
>> A+B
```

```
ans =
```

```
    4    4
```

```
   10    6
```

```
>> B-A
```

```
ans =
```

```
    0    0
```

```
    0   -4
```

MATRIX MATH³¹

- Use the following operators for element by element matrix operations
 - Multiplication - `.*`
 - Division - `./`
 - Exponentiation - `.^`
- Notice the use of the dot (.) before the regular operator
- Indicates element by element operations instead of standard matrix multiplication

`a =` 3 6 5 `b =` 3 5 4

`>> a .* b`

`ans =`

 9 30 20

MATRIX MATH

- Matrix multiplication is not the same as element by element multiplication
- $*$ is the matrix multiplication operator
- Let A have dimensions $m \times n$ and B have dimensions $n \times q$
 - $A * B = C$
 - C has dimensions $m \times q$
- $m \times n * n \times q = m \times q$

inner dimensions
must be equal

Resulting matrix has dimensions
of outer dimensions

MATRIX MULTIPLICATION

- Dot product of rows and columns used to multiply matrices
- Example:

$$\begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix} \times \begin{bmatrix} 12 & 11 \\ 10 & 9 \\ 8 & 7 \end{bmatrix} = \begin{bmatrix} 82 & 73 \\ 112 & 100 \end{bmatrix}$$

$$1*12+3*10+5*8 = 82$$

VECTOR OPERATIONS

- Dot product of vectors $\mathbf{a} = [a_1, a_2, a_3]$ and $\mathbf{b} = [b_1, b_2, b_3]$ defined as $a_1 * b_1 + a_2 * b_2 + a_3 * b_3$

- **dot** is built-in function for this

```
>> a=[1 2 3];
```

```
>> b=[4 5 6];
```

```
>> c=dot(a,b)
```

```
c =
```

```
32
```

- Uses: finding angle between two lines, finding projection on a line, etc.
 - **dot** works for vectors with more than 3 elements

VECTOR OPERATIONS

- Cross product is used extensively in engineering
- Most commonly used to find perpendicular components
 - Finding moments
 - Calculating the vector of a line perpendicular to a surface, etc.
- **cross** is the built-in function
 - Only works with 3-element vectors
 - For 2-D vectors make the third component equal to zero

```
>> a=[1 2 3];           >> d=cross(a,b)
>> b=[4 5 6];           d =
                        -3      6      -3
```

VECTOR OPERATIONS

- Magnitude of vectors is commonly found by engineers
 - Examples
 - To calculate speed from a velocity vector, you find the magnitude of the velocity vector
 - If you have the vector for the line between two points, finding the magnitude of the line vector gives the distance between the points

- Defined as $\sqrt{a_1^2 + a_2^2 + a_3^2}$ for a vector $\mathbf{a} = [a_1, a_2, a_3]$

- **norm** is the built-in function

```
>> line=[2 4 7];  
>> dist=norm(line)  
dist =  
    8.3066
```

VECTOR OPERATIONS

- Matrix inverse
 - Operator $^{-1}$
 - Useful for solving systems of equations
 - Example: Solve the following system of equations for x_1, x_2, x_3

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 = b_1$$

$$a_{21}x_1 + a_{22}x_2 + a_{23}x_3 = b_2$$

$$a_{31}x_1 + a_{32}x_2 + a_{33}x_3 = b_3$$

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}^{-1} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

LOGICAL VECTORS

- Recall: a logical results in a 0 or 1
 - 0 for FALSE
 - 1 for TRUE
- Logicals can be used on vectors to find elements meeting certain requirements
- Example: a vector containing speeds is given. We need to find speeds greater than 55

```
>> speed=[37 65 45 58 23]
```

```
speed =
```

```
    37    65    45    58    23
```

```
>> speed>55
```

```
ans=
```

```
    0    1    0    1    0
```

0 indicates speed at that index is not > 55
1 indicates speed is >55

LOGICAL VECTORS

- Want to pull out the actual values instead of getting 0s and 1s?
 - Use the logical vector from the logical test to index into vector

```
speed=[37 65 45 58 23]
```

```
vals=speed>55
```

```
vals =
```

```
    0    1    0    1    0
```

Logical vector

```
>> hispeed=speed(vals)
```

```
hispeed =
```

```
    65    58
```

LOGICAL FUNCTIONS

40

- **find** – does logical test and returns indices

```
>> speed=[37 65 45 58 23];
```

```
>> hispeed=find(speed>55)
```

```
hispeed =
```

```
2
```

```
4
```

65 located in column 2
58 located in column 4



- **any** – logical test that returns a 1 (true) if ANY elements meet the requirement specified

```
hispeed=any(speed>55)
```

```
hispeed =
```

```
1
```

- **all** – logical test that returns a 1 (true) only if ALL elements meet specified requirement

```
hispeed=all(speed>55)
```

```
hispeed =
```

```
0
```

ISEQUAL

- Need to compare two arrays and get a true/false answer?
- **isequal** - compares two arrays and returns a 0 or 1
 - 0 – arrays not equal
 - 1 – arrays are equal

```
>> accel1=[2.5 5.6 7.3 0.4];
```

```
>> accel2=[2.5 5.7 7.3 0.4];
```

```
>> isequal(accel1,accel2)
```

```
ans =
```

```
0
```



5.6 ≠ 5.7